

FRAME ALIGNER

Verification Plan

SystemVerilog / UVM Functional Verification



Document Information	Value
Project	UVM Frame Aligner Verification
Author	Yonatan Zvida
Document	English Translation of the Original Verification Plan
Tool / Methodology	SystemVerilog, UVM, Synopsys VCS
Revision	September 2026

Purpose

This document defines the planning-stage verification objectives, testbench architecture, and directed test cases for the Frame Aligner DUT. It is an English translation of the original Hebrew verification plan prepared for the project repository.

Translation note: The Hebrew source uses inconsistent signal-name ordering in a few places. This English version uses the interface names `frame_detect` and `fr_byte_position` consistently, while preserving the original verification intent and test-case behavior.

1. DUT Overview and Interfaces

The Frame Aligner receives one byte per cycle and detects valid frame headers in a continuous byte stream. The verification plan focuses on header recognition, synchronization acquisition, synchronization loss, byte-position tracking, and reset behavior.

1.1 DUT Interface

Signal	Direction	Width	Purpose
clk	Input	1 bit	System clock
reset	Input	1 bit	Reset input
rx_data[7:0]	Input	8 bits	One received byte per cycle
fr_byte_position[3:0]	Output	4 bits	Byte position inside the current frame
frame_detect	Output	1 bit	Indicates that frame synchronization is active

1.2 Verification Objectives

1. Verify correct detection of the valid header byte pairs 0xAA -> 0xAF and 0x55 -> 0xBA, and rejection of invalid combinations.
2. Verify that frame_detect rises to 1 only after three consecutive valid frames.
3. Verify that frame_detect falls to 0 after 48 bytes without a detected Header.
4. Verify correct advancement of fr_byte_position on each clock cycle and its reset during state transitions.
5. Verify that reset returns the system to IDLE_FR and clears all counters.

2. Verification Environment Architecture

The planned testbench contains clock and reset generation, a stimulus driver, monitoring, and a scoreboard that checks DUT behavior on every clock cycle.

Component	Responsibility
Clock Generator	Generates a 100 MHz system clock.
Reset	Applies reset to initialize the DUT.
Driver	Drives stimulus on rx_data. Stimulus includes valid frames with legal Headers and Payload, as well as invalid Header scenarios.
Monitor	Samples rx_data and the DUT outputs frame_detect and fr_byte_position on every clock cycle.
Scoreboard	Receives monitored data and performs the expected-behavior checks defined below.

2.1 Scoreboard Checks

1. Detect a valid Header, 0xAA -> 0xAF or 0x55 -> 0xBA, and verify the transition to FR_DATA.
2. Count consecutive valid frames. After three consecutive valid frames, verify that frame_detect rises to 1.
3. While synchronized, if no valid Header is detected for 48 consecutive bytes, verify that frame_detect falls to 0.

3. Directed Test Cases - TC1 to TC5

ID	Test Case	Stimulus / Procedure	Expected Result
TC1	Reset Initialization	Assert reset for several clock cycles at the beginning of the run.	The FSM enters IDLE_FR. fr_byte_position, legal_frame_counter, and na_byte_counter are cleared to 0, and frame_detect is set to 0.
TC2	Remain in IDLE Without Valid Information	Send random rx_data values that do not match a valid Header.	The DUT remains in IDLE_FR and does not begin counting valid frames.
TC3	Detect Header Type A - 0xAA 0xAF	Send 0xAA as the LSB, immediately followed by 0xAF as the MSB.	The DUT recognizes a valid Header, the FSM advances to DATA_FR, and fr_byte_position begins tracking the byte location inside the frame.
TC4	Detect Header Type B - 0x55 0xBA	Send 0x55 as the LSB, immediately followed by 0xBA as the MSB.	The DUT recognizes a valid Header and transitions cleanly to FR_DATA.
TC5	Reject an Invalid Header	Send a valid LSB such as 0xAA, followed by an invalid matching byte such as 0xBA or 0x00.	The DUT starts the detection process and reaches HLSB_FR, but fails the MSB check. It does not enter DATA_FR, returns to IDLE_FR, and legal_frame_counter does not increment.

4. Directed Test Cases - TC6 to TC10

ID	Test Case	Stimulus / Procedure	Expected Result
TC6	Full Synchronization Lock	Send three consecutive valid frames. Each frame contains a valid Header and a full-length Payload.	When the Header of the third frame is detected, legal_frame_counter reaches 3 and frame_detect rises immediately to 1.
TC7	Interrupt the Synchronization Sequence	Send two valid frames, raising legal_frame_counter to 2, then send a third frame with a broken or invalid Header.	legal_frame_counter resets to 0 and frame_detect remains low at 0.
TC8	Track the Byte Position	Monitor fr_byte_position while receiving a valid frame in FR_DATA.	fr_byte_position increments by 1 on every clock cycle and resets according to the FSM state transitions.
TC9	Loss of Synchronization	Bring the DUT to full synchronization, frame_detect = 1, then send bytes that are not Headers on rx_data.	na_byte_counter increments. After 48 consecutive bytes without a Header, when the counter reaches 47, frame_detect falls to 0.
TC10	Recovery After Synchronization Loss	Bring the DUT to the synchronization-loss condition, then immediately send three new valid frames.	The DUT recovers, legal_frame_counter climbs back to 3, and frame_detect returns to 1.

5. Planned Completion Criteria

- All directed test cases execute without testbench infrastructure failures.
- Valid and invalid Header behavior is checked against the expected state progression.
- Synchronization acquisition, loss, and recovery behavior is observed and checked.
- Byte-position tracking and reset behavior are verified cycle by cycle.

Source: Original Hebrew Verification Plan, translated for the GitHub project documentation.